

Plano de Correções para Estabilizar Queda/ Restaurar e Melhorar Segurança/Desempenho

Prioridade: alta → baixa

Fase 0: Congelar instabilidade

Objetivo: evitar variáveis novas durante os testes.

Ações:

1. Pausar mudanças em schemas e policies durante os testes.
2. Testar fluxo mínimo no app:
3. Login
4. Uma consulta simples
5. Conexão Realtime via websocket por 5 a 10 minutos, se aplicável
6. Registrar o horário exato da queda/restauração.
7. Depois, correlacionar com:
8. Deploys
9. Uploads
10. Tarefas agendadas
11. Migrações
12. Eventos do Realtime

Entrega esperada: janela de teste estável e dados mínimos para correlação.

Fase 1: Reduzir lentidão em RLS

Diagnóstico: o projeto tem sinais fortes de custo alto em RLS, incluindo:

- Policies permissivas
- Linter indicando funções de autenticação reavaliadas por linha
- Possíveis policies sempre true em escrita
- Múltiplas policies permissivas para mesma ação/função
- FKs sem índice

Objetivo: reduzir custo por consulta e evitar timeouts no pool, principalmente no Realtime.

Checklist:

1. Padronizar policies para usar:

```
(select auth.uid())
```

Em vez de:

```
auth.uid()
```

Exemplo:

```
(select auth.uid()) = id
```

1. Corrigir tabelas marcadas com `auth_rols_initplan`, como:

2. `users`

3. `roles`

4. `jobs`

5. `candidates`

6. `feedbacks`

7. Remover policies sempre true em operações de escrita:

8. `INSERT`

9. `UPDATE`

10. `DELETE`

11. `ALL`

Tabelas citadas pelo linter:

- `assignments`
- `admin_actions`
- `schedule_snapshots`

- Reduzir múltiplas policies permissivas para a mesma role/ação.

Exemplos de tabelas afetadas:

- `assignments`
- `candidates`
- `candidate_stage_history`

Meta: manter uma regra principal por operação ou reorganizar as regras para reduzir permissividade.

1. Criar índices para foreign keys sem cobertura.

Motivo: FKs sem índice pioram planos de execução e aumentam custo quando RLS usa `EXISTS`, joins ou subqueries.

Entrega esperada: Realtime menos sujeito a timeouts, 521 e quedas.

Fase 2: Corrigir vulnerabilidade de autorização

Risco crítico identificado:

- RLS referencia `user_metadata` em `public.users`
- Verificação de admin baseada em `user_metadata`

Problema: `user_metadata` não deve ser fonte confiável para autorização.

Objetivo: remover dependência de `user_metadata` em regras de acesso.

Correção recomendada:

```
EXISTS (  
  SELECT 1  
  FROM public.users u  
  WHERE u.id = (select auth.uid())  
         AND u.role = 'admin'  
)
```

Alternativas:

1. Usar tabela interna confiável, como `public.users`.
2. Usar `app_metadata`, se o fluxo de gestão for seguro.
3. Nunca usar `user_metadata` para autorização sensível.

Ações:

1. Trocar admin checks baseados em `user_metadata`.
2. Revisar policies relacionadas à tabela `users`.
3. Reduzir sobreposição de policies.

Entrega esperada: menor risco de bypass e menos retrabalho em auditoria.

Fase 3: Endurecer funções e exposição via API

Objetivo: reduzir superfície de ataque e exposição indevida por RPC.

Problemas detectados:

1. Funções `SECURITY DEFINER` acessíveis por `anon` ou `authenticated`.

2. Funções em schema `public` expostas via RPC.
3. `search_path` mutável em views/funções.

Funções citadas:

- `get_user_role`
- `get_user_tenant`
- `handle_new_user`

Ações recomendadas:

1. Revogar `EXECUTE` de `anon` e `authenticated` quando a função não deve ser pública:

```
REVOKE EXECUTE ON FUNCTION public.nome_da_funcao FROM anon;  
REVOKE EXECUTE ON FUNCTION public.nome_da_funcao FROM authenticated;
```

1. Avaliar troca de `SECURITY DEFINER` para `SECURITY INVOKER`, quando possível.
2. Mover funções sensíveis para schema não exposto.
3. Definir `search_path` explicitamente nas funções.

Exemplo:

```
SET search_path = public, pg_temp;
```

Entrega esperada: menos exposição via RPC e menor risco de execução com privilégios indevidos.

Fase 4: Validação final

Objetivo: confirmar estabilidade, segurança e redução de alertas.

Checklist:

1. Rodar o linter novamente.
2. Validar redução ou eliminação de:
3. `rls_references_user_metadata`
4. `rls_policy_always_true`
5. `multiple_permissive_policies`
6. `auth_rls_initplan`
7. FKs sem índice
8. Testar Realtime por 15 a 30 minutos.
9. Monitorar:
10. Erros 521

11. Timeouts
12. Quedas/restaurações
13. Latência de queries
14. Uso do pool
15. Logs do Realtime

Entrega esperada: ambiente mais estável, seguro e auditável.

Ordem de prioridade recomendada

1. Congelar mudanças e reproduzir queda.
 2. Corrigir RLS de alto custo.
 3. Remover autorização baseada em `user_metadata`.
 4. Endurecer funções expostas via RPC.
 5. Reexecutar linter e validar Realtime.
-

Próximo passo

Para transformar isso em tarefas pendentes, preciso definir a prioridade entre:

1. Estabilizar Realtime primeiro.
2. Corrigir segurança primeiro.

Também preciso identificar quais tabelas mais pesam no app atualmente, por exemplo:

- `users`
- `candidates`
- `candidate_stage_history`
- `assignments`
- `jobs`
- `feedbacks`

Critério recomendado: priorizar primeiro as tabelas mais consultadas em telas críticas e as mais usadas pelo Realtime.